A Comparative Study of Value-Based and Policy-Based Reinforcement Learning on CartPole-v1

Het Jethva
FEST
Adani University
ECSDT24403
hetjethva07@gmail.com

Foram Amin
FEST
Adani University
ECSDT24403
foramamin107@gmail.com

Abstract—We compare tabular Q-learning, a deep Q-network (DQN), and REINFORCE with a learned baseline on CartPole-v1 under a controlled protocol: a single shared training loop, an identical budget of 100k environment steps, a held-out evaluation set fixed across all methods, an equal hyperparameter search budget per method on seeds disjoint from those reported, and 10 evaluation seeds per configuration aggregated by interquartile mean with bootstrap intervals. Because all three methods solve the task, we do not rank them; we measure what each one spends to succeed, across sample efficiency, computation, reliability and implementation cost. The orderings disagree. REINFORCE reaches the solved threshold first (18k steps), ends at the maximum return (500), and uses 470 gradient updates—two orders of magnitude fewer than DQN's 99k—with the fewest hyperparameters of the three. DQN is the most expensive method on every cost axis and leads on none. Tabular Q-learning is the fastest in wall-clock time and the least reliable: every seed produces a maximum-return evaluation episode, yet its final interquartile mean is 226, because the discretisation its exact representation requires makes distinct states indistinguishable. Ablations locate these outcomes precisely: removing DQN's target network eliminates learning entirely (9), whereas removing REINFORCE's baseline is costly but survivable (361). We conclude that on this task, algorithmic machinery is worth its cost only in proportion to the difficulty it was designed to address, and we report all 233 runs.

Index Terms—reinforcement learning, Q-learning, deep Q-network, policy gradient, function approximation, empirical comparison

I. Introduction

CartPole [1] is the standard first problem in reinforcement learning. It is small, it is fast, and every general-purpose method solves it. That last property is what makes it interesting for a comparison: because the outcome is not in doubt, the question stops being which method works and becomes what each method costs to make it work.

This paper compares three methods that sit at three levels of approximation:

- Tabular Q-learning [2], which represents the value function exactly over a discretised state space;
- DQN [3], which replaces the table with a neural network and adds the two devices—a replay buffer and a target network—that make that replacement stable;
- REINFORCE with a learned baseline [4], which abandons the value function as the object of interest and optimises the policy directly.

The comparison is not a ranking. All three methods reach a working policy on CartPole-v1, so a claim of the form "method X is best" would be both uninformative and already known. The contribution here is a trade-off map: a measurement of what each method spends to arrive at that policy, along four axes that are usually reported separately or not at all—environment samples, computation, reliability across random seeds, and implementation effort.

Two design commitments make the resulting numbers comparable. First, every method runs under a single shared training loop with an identical budget of 100k environment steps and an identical evaluation procedure, so no method can gain an advantage from the harness. Second, hyperparameters are selected by a random search with an equal trial budget per method, on seeds disjoint from those used for the reported results. Without the second commitment, a comparison paper measures how the author distributed their attention.

The contributions are:

1) A controlled comparison of the three methods on CartPole-v1 over 10 seeds each, under a shared budget and a fixed evaluation set.
2) A trade-off table quantifying sample cost, compute cost, reliability and implementation cost together, including the implementation-cost axis that empirical RL papers normally omit.
3) A sensitivity study for each method over the hyperparameter that carries its characteristic failure mode, and ablations of the two components that distinguish the deep methods from their textbook forms.
4) A complete, seed-reproducible implementation with the full result set.

II. Background

A. Problem setting

Reinforcement learning problems are modelled as a Markov decision process $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$, where $P(s' \mid s, a)$ is the transition distribution, $r(s, a)$ the reward, and $\gamma \in [0, 1)$ a discount factor. The agent seeks a policy $\pi(a \mid s)$ maximising the expected discounted return $\mathbb{E}[\sum_t \gamma^t r_t]$ [5].

CartPole-v1 has a four-dimensional continuous observation—cart position, cart velocity, pole angle and pole angular velocity—and two actions, pushing the cart left or right. Every timestep yields a reward of 1. An episode terminates when the pole angle exceeds 12° or the cart leaves the track, and is truncated at 500 steps. The maximum achievable return is therefore 500, and the conventional definition of solving the task is a mean return of at least 475.

B. Tabular Q-learning

Q-learning [2] estimates the optimal action-value function by the off-policy update

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right], \tag{1}$$

which converges to Q^* under standard conditions when every state–action pair is visited infinitely often and Q is represented exactly.

That last requirement is the difficulty here. An exact representation needs a finite state space, and CartPole's is continuous, so the observation must be discretised into bins. Two of its four dimensions—the two velocities—are unbounded, so they must additionally be clipped before they can be binned. The resulting agent therefore does not solve CartPole; it solves a discretised approximation of CartPole, and the gap between the two is a cost that the method pays and the other two do not.

C. Deep Q-networks

DQN [3] replaces the table with a parameterised function $Q_\theta(s, a)$ and minimises the temporal-difference error against a bootstrap target. Two devices stabilise what is otherwise a moving-target regression:

- a replay buffer, which stores past transitions and samples training batches uniformly from them, breaking the temporal correlation that would otherwise violate the i.i.d. assumption of stochastic gradient descent;
- a target network Q_{θ^-}, a periodically refreshed copy of the online network used to compute the bootstrap term, which stops the regression target from moving with every gradient step.

The loss on a sampled transition is

$$\mathcal{L}(\theta) = \ell_\delta \Big(Q_\theta(s, a) - \Big[r + \gamma (1 - d) \max_{a'} Q_{\theta^-}(s', a') \Big] \Big), \tag{2}$$

where ℓ_δ is the Huber loss and d indicates a genuine termination. Section VI ablates the target network to show what it buys.

D. REINFORCE with a baseline

REINFORCE [4] optimises a parameterised policy π_θ directly, following the policy gradient

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} \left[\nabla_\theta \log \pi_\theta(a_t \mid s_t) \, (G_t - b(s_t)) \right], \tag{3}$$

where G_t is the Monte-Carlo return from timestep t and $b(s_t)$ is a baseline. Any function of the state alone leaves the gradient unbiased, so the baseline is free to reduce variance; the standard choice, adopted here, is a learned state-value estimate $\hat{V}_\phi(s)$ trained by regression on the observed returns [5].

The baseline is not cosmetic. Because every CartPole reward is +1, the raw return G_t is large and strictly positive, so an unbaselined update increases the log-probability of every action taken in a successful episode, including the poor ones—the gradient carries the correct signal only through differences in magnitude. Subtracting $\hat{V}_\phi(s)$ centres the signal so that only better-than-expected actions are reinforced. This is ablated in Section VI.

Note that returns are deliberately not whitened. Dividing by the batch standard deviation, common in tutorial implementations, rescales the gradient as well as centring it; it is an adaptive step size rather than a baseline, and it would confound the ablation.

III. Experimental Design

The design follows the reproducibility recommendations of Henderson et al. [6] and the statistical recommendations of Agarwal et al. [7].

A. Shared budget and a single training loop

All three agents run under one training loop, differing only in what they do with a transition. Each run receives exactly 100k environment steps, and learning curves are plotted against environment steps rather than episodes. The distinction matters: a CartPole episode lasts around 20 steps for an untrained policy and 500 for a good one, so an episode budget silently grants the better method up to twenty-five times more data.

The loop bootstraps through a time-limit truncation but not through a genuine termination. Treating the 500-step truncation as terminal would teach both value-based methods that balancing the pole indefinitely is worth nothing.

B. Evaluation protocol

Training return is a poor measure of policy quality, because it includes the cost of exploration and therefore penalises whichever method explores most. Every 2k steps, training is paused and the agent is evaluated for 10 episodes with exploration disabled: greedy action selection for the two value-based methods, and the mode of the action distribution for REINFORCE.

The evaluation episodes use a fixed set of environment seeds, identical for every method, every seed and every evaluation point. The evaluation set is therefore held out and constant, so a difference between two curves reflects the policies rather than a difference in the states they happened to be tested on.

C. Equal-budget hyperparameter search

Each method receives 12 random-search trials drawn from a discrete space of its own hyperparameters, evaluated on 3 tuning seeds and scored by area under the evaluation curve—a criterion that rewards reaching a high return and reaching it early. The winning configuration is then frozen and run on 10 fresh evaluation seeds. Tuning seeds and evaluation seeds are disjoint, so no reported result is tuned on its own test set. The full search spaces and the selected values appear in Appendix A.

D. Statistics

Reinforcement learning results across seeds are skewed and frequently bimodal: a run either solves the task or collapses. The sample mean describes neither mode, and one failed seed in ten moves it substantially. Following Agarwal et al. [7], the point estimate is the interquartile mean (IQM), the mean of the middle 50% of the seeds, and uncertainty is reported as a 95% percentile bootstrap interval resampled over seeds. Per-seed final performance is also plotted directly, so that any bimodality remains visible rather than being absorbed into an aggregate.

IV. Implementation

The implementation is a single Python package. All three agents expose the same four-operation interface—select an action, accept a transition, finish an episode, report diagnostics—so the training loop, the evaluation procedure and the logging are written once and shared. Every behavioural difference between the methods lives in the update.

Both neural methods use a two-layer multilayer perceptron with 128 hidden units per layer and ReLU activations, optimised with Adam. DQN uses this network for Q_θ and a periodically synchronised copy for Q_{θ^-}; REINFORCE uses one for the policy logits and a second, separately optimised network for the baseline $\hat{V}_\phi$.

One configuration object fully determines one run, and a hash of it names that run's output files. This makes the run matrix resume-safe—an interrupted sweep restarts without repeating completed work—and it means every curve in this paper can be traced to the exact settings that produced it. Runs execute in a process pool, one run per worker, with one PyTorch compute thread per worker to avoid nested thread pools. The reported rerun uses eight workers on an AMD Ryzen 7 8845HS running Windows 11, with Python 3.12.14 and PyTorch 2.13.0. Wall-clock times include held-out evaluation and reflect this concurrent execution setup.

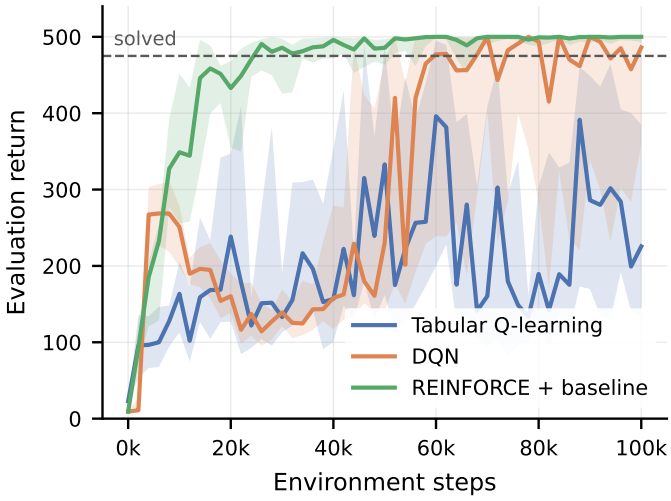


Fig. 1. Evaluation return against environment steps. Lines are the interquartile mean over 10 seeds; shaded bands are 95% bootstrap intervals. The dashed line marks the solved threshold of 475.

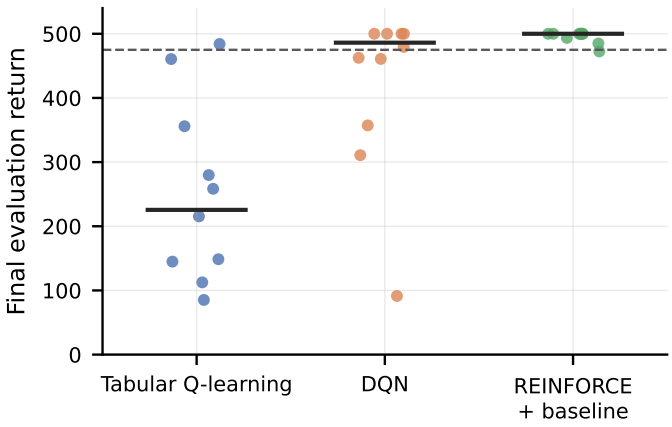


Fig. 2. Final evaluation return of every individual seed. Horizontal bars mark the interquartile mean. The spread of the tabular agent, and the single failed DQN seed, are invisible in any aggregate statistic.

Correctness is protected at the points where an error would be silent rather than loud. A wrong sign in the advantage or an off-by-one in the discounted return does not raise an exception; it produces a plausible learning curve and a wrong paper. The discounted-return computation, the state discretiser's clipping behaviour, the replay buffer's contents under overwrite, and the aggregation statistics are therefore checked against hand-computed values, and every method is checked to reproduce its curve exactly from a fixed seed.

Every figure and table in this paper is generated from the raw result files, and every number quoted in the text is a macro produced by the same stage, so the prose cannot drift out of step with the data.

TABLE I
Trade-off summary over 10 seeds per method, under an identical budget of 100k environment steps. Steps-to-solve is the median over the seeds that reached the threshold. Final return is the interquartile mean with a 95% bootstrap interval. Implementation cost counts executable source lines in the agent module and the number of tunable hyperparameters; the shared training loop is excluded because it is identical for all three methods.

Method	Steps to solve	Final return (IQM [95% CI])	Wall clock (s)	Gradient updates	Failed seeds	LoC / hyperparams
Tabular Q-learning	36k	226 [133, 390]	16	100k	10%	87 / 7
DQN	55k	486 [354, 500]	180	99k	0%	136 / 12
REINFORCE + baseline	18k	500 [493, 500]	39	470	0%	113 / 6

V. Results

Fig. 1 shows the learning curves and Table I summarises the comparison. The headline observation is that the ordering of the three methods depends entirely on which cost you measure, and no method leads on more than a subset of the axes.

A. Sample efficiency

REINFORCE with a baseline reaches the solved threshold first, at a median of 18k environment steps, against 55k for DQN. This is the reverse of the usual expectation—replay-based value methods are normally the sample-efficient choice, because each transition is reused many times—and it is a consequence of the task rather than of the algorithms. CartPole's reward is dense and its episodes are short, so a Monte-Carlo return is a low-variance learning signal available almost immediately, and the bias-free policy gradient has little to overcome. DQN spends its early budget filling a replay buffer and waiting for a bootstrapped value estimate to become self-consistent.

The tabular agent is the more interesting case. It first touches the threshold at a median of 36k steps—earlier than DQN—but this is not evidence that it learns a better policy. It is evidence that the threshold is a transient event for this method rather than a stable state, which the next subsection makes explicit.

B. Final performance and reliability

At the end of the budget, REINFORCE attains an interquartile mean of 500 (95% CI [493, 500]), that is, the maximum return the environment permits, on every seed in the interquartile range. DQN attains 486 [354, 500]. The tabular agent attains 226 [133, 390]—less than half.

Fig. 2 shows why the aggregate alone would be misleading. The tabular agent's per-seed final returns are spread across almost the whole available range, despite every seed having produced an individual evaluation episode scoring 500 during training. The agent does not converge to a mediocre policy; it oscillates between good and poor ones, and the final measurement catches it at an arbitrary point in that oscillation. This is the expected behaviour of Q-learning under function approximation coarse enough to violate its convergence conditions: with the selected discretisation of 1 024 cells, distinct physical states share a table entry, so no fixed point of the tabular update corresponds to a good policy in the underlying continuous problem.

Supporting this, only 32% of the table's cells are ever visited within the budget, even though the search selected a relatively coarse one of its candidate discretisations. Finer grids were available and were rejected: a finer grid represents the dynamics better but cannot be filled with 100k samples. That tension—representational fidelity against sample cost—is the tabular method's characteristic trade-off, and Section VI measures it directly.

DQN reaches the threshold on all 10 seeds, but one seed ends the budget far below its own peak, which is what pulls its final interquartile mean below REINFORCE's. This is the familiar instability of bootstrapped value learning: performance is not monotonic, and a policy that has been solved can be unlearned.

C. Computational cost

The compute ordering is different again, and it separates into two distinct quantities that the wall-clock column alone would conflate.

Measured in gradient updates, REINFORCE is cheaper than DQN by more than two orders of magnitude: 470 against 99k. DQN performs one minibatch update per environment step by construction, whereas REINFORCE performs one update per completed episode—and as its policy improves, its episodes lengthen to 500 steps, so it updates less often the better it gets. Its compute cost per unit of experience falls during training, which is unusual and worth noting.

Measured in wall-clock seconds the tabular agent is cheapest at 16 s, against 39 s and 180 s, because a table update is a single arithmetic operation with no gradient and no framework overhead. It performs the most updates of any method—one per step—and is still by far the fastest.

D. Implementation cost

The final column of Table I is the axis empirical comparisons usually omit. Counting executable source lines in the agent module and the number of tunable hyperparameters, the tabular agent costs 87 lines and 7 hyperparameters, REINFORCE 113 lines and 6, and DQN 136 lines and 12.

DQN is the most expensive method to implement and to configure, on a task where it finishes second in sample efficiency and second in final return. Its replay buffer, its target network and the six hyperparameters that govern them exist to solve problems—correlated samples, moving regression targets, high-variance returns over long horizons—that CartPole does not pose severely enough to justify the cost.

VI. Extended Analysis

This section reports work beyond the originally proposed scope: a sensitivity sweep for each method over the hyperparameter that carries its characteristic failure mode, and an ablation of each stabilising component in the two deep methods. Sweeps use 5 seeds per setting; ablations reuse the full 10 evaluation seeds.

A. Sensitivity

Fig. 3 shows that the three methods are fragile along entirely different axes, and to very different degrees.

Tabular Q-learning is sensitive to discretisation, and not monotonically so. Final interquartile means range from 131 at the coarsest grid (324 cells) to 401 at the finest (82,944 cells), a spread of 270 return. The relationship is not, however, ordered: the 1,024-cell grid selected by the search reaches 253 with no failed seeds, while the intermediate 5,184- and 16,384-cell grids reach only 153 and 170, and the 16,384-cell grid fails on 60% of its seeds.

Two competing effects produce this shape. A finer grid represents the dynamics more faithfully, which raises the ceiling; but it also spreads a fixed sample budget over more cells, which slows learning and leaves most of the table unvisited. The coarsest grid is bad for the first reason and the intermediate grids are bad for the second. The finest grid recovers because CartPole's start states are tightly clustered, so a fine grid concentrated on the small region the agent actually visits behaves like a much smaller table—at the cost of a 40% seed failure rate. The search's choice was not the highest-scoring setting but the most reliable one, which is the correct selection under an area-under-curve criterion.

DQN is sensitive to target-network staleness, with an interior optimum. The spread is the largest of the three at 408 return. Synchronising the target network every update destroys learning outright (final return 9, no seed ever solving the task), because it removes the separation between prediction and target that makes the regression well posed. Synchronising every 500 updates is best (418, no failed seeds). Intervals that are too long are also harmful but less catastrophically: at 1,000 and 5,000 updates the agent still learns, from a target that is increasingly out of date, and fails on 40% of seeds.

REINFORCE is the least sensitive of the three. Across two orders of magnitude of learning rate the final return varies by only 41, and four of the five settings end above the solved threshold. What the learning rate changes is speed and reliability rather than eventual quality: 10^{-4} solves at 50k steps, 10^{-3} at 6k, and 3×10^{-3} at 4k but with a lower final return, while 10^{-2} reaches a high final return yet fails on 40% of seeds.

This last panel also exposes a limitation of the search rather than of the method. The selected learning rate of 3×10^{-4} solves at 20k steps, whereas 10^{-3}—also in the search space—solves at 6k with an identical final return. With 12 trials over a joint space of four hyperparameters, the search sampled a good configuration rather than the best available one. The reported REINFORCE result is therefore, if anything, conservative.

TABLE II
Component ablations. Each removal is a one-flag change from the tuned configuration, run on the same 10 evaluation seeds.

Arm	Final return (IQM [95% CI])	Steps to solve	Failed seeds
REINFORCE + baseline	500 [493, 500]	18k	0%
without baseline	361 [172, 500]	38k	20%
DQN	486 [354, 500]	55k	0%
without target network	9 [9, 10]	—	100%

B. Ablations

Fig. 4 and Table II isolate what each stabilising component contributes.

The target network is not an optimisation for DQN; it is a precondition. Removing it drops the final interquartile mean from 486 to 9—barely above the return of a random policy—and no seed reaches the threshold at any point in training. Without it, the network's own prediction supplies the regression target, so improving the prediction moves the target, and the two chase each other instead of converging.

This result also serves as an internal consistency check. The ablation and the interval = 1 arm of the DQN sweep implement the same thing through different code paths—one disables the target network, the other refreshes it every update—and they agree to within a point of final return (9 against 9). Agreement between two independent routes to the same condition is evidence that neither is an artefact.

The baseline is a substantial improvement for REINFORCE, but not a precondition. Removing it lowers the final interquartile mean from 500 to 361, delays the median solve from 18k to 38k steps—more than double—and raises the seed failure rate from 0% to 20%. The confidence interval widens dramatically, which is the signature the theory predicts: the baseline is a variance-reduction device, so removing it should degrade consistency more than it degrades the best case, and Fig. 4 shows exactly that shape—a band that spans nearly the full range of possible returns.

The asymmetry between the two ablations is itself informative. DQN's target network addresses a structural defect that makes the objective ill-posed, so its removal

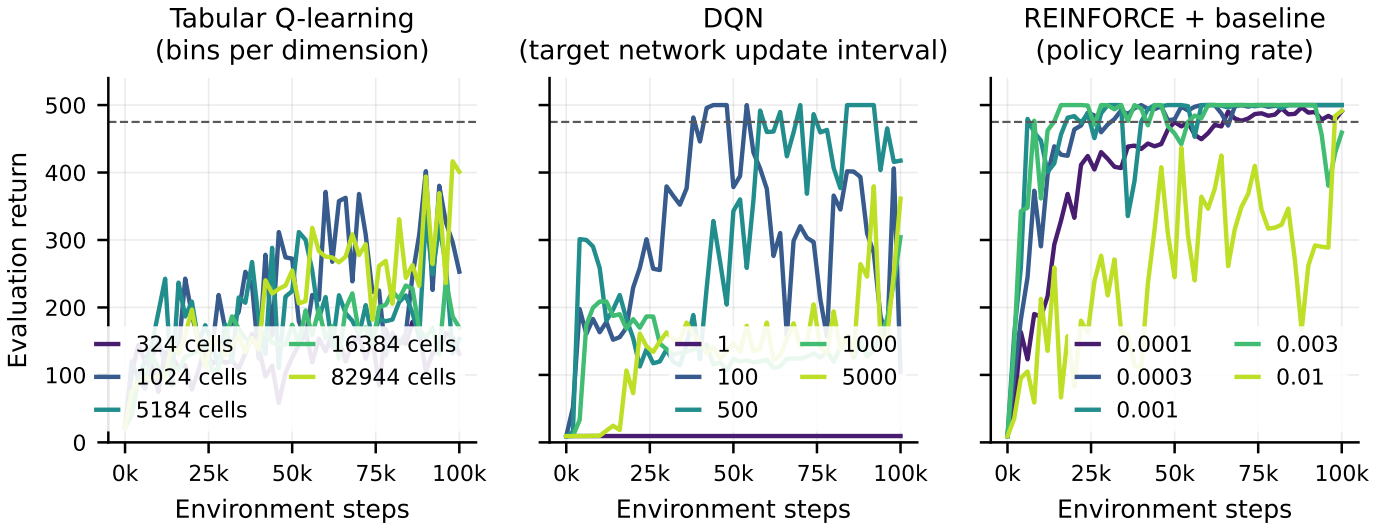


Fig. 3. Sensitivity sweeps. Each panel varies one hyperparameter of one method and holds the rest at their selected values. Lines are interquartile means over 5 seeds.

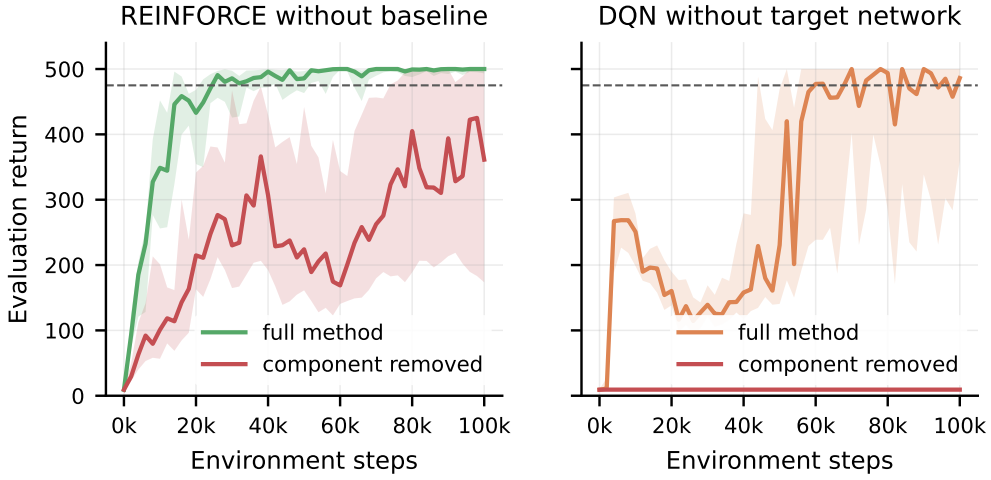


Fig. 4. Component ablations against the full method. Removing DQN's target network eliminates learning entirely; removing REINFORCE's baseline slows learning and widens the spread across seeds.

is fatal. REINFORCE's baseline addresses variance in an objective that is already well-posed and unbiased, so its removal is expensive but survivable.

VII. Discussion

A. There is no single winner, but there is a pattern

Across the four axes measured, no method leads on all of them. The tabular agent is the cheapest in wall-clock time and among the cheapest to implement, and it is by a wide margin the least reliable. DQN is the most expensive method on every cost axis—implementation, configuration, gradient updates, wall-clock—and it does not lead on any performance axis. REINFORCE with a baseline reaches the threshold soonest, ends highest, and uses the fewest updates and the fewest hyperparameters, but it is a Monte-Carlo method whose advantages here rest on properties of the task rather than on generality.

The pattern is that method complexity should be matched to task difficulty rather than assumed to dominate it. Each of DQN's components addresses a specific failure: the replay buffer addresses correlated samples, the target network addresses a moving regression target, and both are indispensable on the high-dimensional, long-horizon problems the method was designed for [3]. CartPole poses neither problem severely. The machinery is therefore paid for and largely unused, which is exactly what the trade-off table shows.

B. Why the expected ordering reverses

Textbook intuition predicts that a replay-based value method is more sample efficient than a Monte-Carlo policy gradient, because replay reuses each transition many

times while REINFORCE discards each episode after one update. That intuition assumes the Monte-Carlo return is a noisy signal. On CartPole it is not: the reward is dense, every episode terminates within 500 steps, and the return is therefore a bounded, low-variance estimate available at the end of every episode. With a learned baseline removing most of the remaining variance, the unbiased policy gradient simply has little to gain from bootstrapping—and it avoids bootstrapping's cost, which is a value estimate that must become self-consistent before it is useful.

The result should therefore be read as a statement about the interaction between a method and a task, not as a ranking of the methods. It predicts that the ordering would change under a sparse reward or a long horizon, where the Monte-Carlo return becomes high-variance and bootstrapping earns its cost.

C. Discretisation is a modelling decision, not a preprocessing step

The tabular results make the clearest single point in this study. Q-learning is guaranteed to converge to the optimal action-value function under an exact representation [2]. Applying it to CartPole requires abandoning that condition, and the consequence is not a slightly worse policy—it is the loss of a stable one. Every seed produces a maximum-return evaluation episode, but none ends with a maximum mean evaluation return, because the discretisation makes states that require different actions indistinguishable.

Choosing the bin count is therefore not a tuning detail on the same footing as a learning rate. It defines what problem is being solved. The search selected the grid with the highest mean tuning area under the curve under a 100k step budget and simultaneously the choice that guarantees the representation is inadequate. There is no setting that escapes both horns within this budget, which is precisely the argument for function approximation.

D. Practical reading

For a practitioner facing a low-dimensional, densely rewarded control problem, the evidence here favours starting with a policy gradient and a learned baseline: it was the simplest of the three to implement, needed the fewest hyperparameters, required the least tuning effort to work, and performed best. Reaching for a replay-based value method should follow evidence that the task demands it—long horizons, sparse reward, or expensive environment interaction that makes sample reuse worth its overhead—rather than preceding that evidence.

VIII. Threats to Validity

A single environment. Every result here comes from CartPole-v1. The task is low-dimensional, densely rewarded, and solvable by a linear policy, and it is therefore the most favourable possible setting for the tabular method: four dimensions is few enough to discretise at all. On a task with a higher-dimensional observation, the table would grow exponentially while the two neural methods would be unchanged in size. The direction of that effect is predictable, but its magnitude is not measured here, and no result in this paper should be read as a claim about reinforcement learning in general.

Dense reward. CartPole gives a reward at every timestep, which suits Monte-Carlo policy gradients particularly well. Under a sparse reward, REINFORCE receives an informative gradient only from the rare successful episode, and the comparison could reverse. Testing that would need a second environment.

Search budget. The hyperparameter search is equal across methods by construction, but it is small—12 trials over a discrete grid. A larger budget could favour whichever method has the most delicate configuration, most plausibly DQN, which has twice as many hyperparameters as REINFORCE. The equality of the budget is defensible; its size is a limitation.

Ten seeds. 10 seeds is enough for an interquartile mean to be meaningful and enough to expose a bimodal outcome, but it is too few for well-powered significance testing between arms whose intervals overlap. No pairwise significance claim is made anywhere in this paper for that reason.

Implementation cost is partly subjective. Source lines and hyperparameter count are objective once the counting rule is fixed, but the rule involves judgement: the shared training loop is excluded, and a differently factored implementation would produce different numbers. The column is best read as an ordering, not as a measurement.

Wall-clock time is machine-specific. All runs executed on the same machine, one thread each, so the comparison between methods is fair; the absolute values are not portable, and the ratio would change on a GPU—where the two neural methods would gain and the tabular method would not.

IX. Conclusion

This paper compared tabular Q-learning, DQN, and REINFORCE with a learned baseline on CartPole-v1 under a shared budget of 100k environment steps, a fixed held-out evaluation protocol, an equal hyperparameter search budget per method, and 10 evaluation seeds per configuration, reporting interquartile means with bootstrap intervals over 233 runs in total.

The comparison does not produce a ranking, because the ordering of the three methods changes with the cost being measured. Tabular Q-learning is the cheapest to run and the least reliable: every seed produces a maximum-return evaluation episode, but none ends with a maximum mean evaluation return, because the discretisation its exact representation demands destroys the distinctions the task requires. DQN is the most expensive method to implement, to configure and to compute, and on this task

it leads on no performance axis; its stabilising machinery addresses difficulties CartPole does not present. REINFORCE with a baseline reaches the threshold soonest, ends at the maximum return, and does so with the fewest hyperparameters and two orders of magnitude fewer gradient updates than DQN.

The ablations locate these outcomes in specific components rather than in the methods as wholes, and the sensitivity sweeps show that each method's characteristic fragility lies on a different axis: representational granularity for the tabular agent, target-network staleness for DQN, and step size for REINFORCE.

The central limitation is the single environment. CartPole's low dimensionality is what makes a table feasible at all, and its dense reward is what makes Monte-Carlo returns cheap; both conditions favour the methods that performed best here. The natural next step is to repeat this protocol on a task that breaks one condition at a time—a higher-dimensional observation to stress the table, and a sparse reward to stress the Monte-Carlo return—which would test whether the trade-off structure reported here is a property of the methods or of CartPole.

References

[1] A. G. Barto, R. S. Sutton, and C. W. Anderson, "Neuronlike adaptive elements that can solve difficult learning control problems," IEEE Transactions on Systems, Man, and Cybernetics, vol. SMC-13, no. 5, pp. 834–846, 1983.

[2] C. J. C. H. Watkins and P. Dayan, "Q-learning," Machine Learning, vol. 8, no. 3–4, pp. 279–292, 1992.

[3] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," Nature, vol. 518, no. 7540, pp. 529–533, 2015.

[4] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," Machine Learning, vol. 8, no. 3–4, pp. 229–256, 1992.

[5] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA: MIT Press, 2018.

[6] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, "Deep reinforcement learning that matters," in Proceedings of the AAAI Conference on Artificial Intelligence, vol. 32, no. 1, 2018, pp. 3207–3214.

[7] R. Agarwal, M. Schwarzer, P. S. Castro, A. Courville, and M. G. Bellemare, "Deep reinforcement learning at the edge of the statistical precipice," in Advances in Neural Information Processing Systems, vol. 34, 2021, pp. 29 304–29 320.

Appendix A
Hyperparameter Settings

TABLE III
Complete hyperparameter settings. Values marked ⋆ were selected by the equal-budget random search (12 trials per method, 3 tuning seeds each); the rest are fixed defaults shared by every arm.

Parameter	Value	Tuned
Tabular Q-learning		
alpha	0.5	⋆
bins	(4, 4, 8, 8)	⋆
clip_velocity	(3.0, 3.5)	
eps_decay_steps	10000	⋆
eps_end	0.05	
eps_start	1.0	
gamma	0.99	⋆
DQN		
batch_size	32	⋆
buffer_size	50000	
eps_decay_steps	10000	⋆
eps_end	0.05	
eps_start	1.0	
gamma	0.99	
grad_clip	10.0	
hidden	(128, 128)	
learning_starts	1000	
lr	0.0003	⋆
target_update_interval	500	⋆
train_frequency	1	
use_target_network	True	
REINFORCE + baseline		
baseline_lr	0.003	⋆
episodes_per_update	1	⋆
gamma	0.999	⋆
grad_clip	10.0	
hidden	(128, 128)	
lr	0.0003	⋆
use_baseline	True	